

31/10/2025 - Laboratorio de SO (Ude@)

Lab 3: Desde: 31/10/2025 - Hasta: 21/11/2025 (3 semanas)

Lab 4:

Nombre de la clase	Título del curso	Estudiantes	Rango de fechas	Etiqueta
DO188-LATAM-UDA00012	DO188 - Red Hat OpenShift Development I: Introduction to Containers with Podman 4.18	20	Sep 07 2025 - Dec 08 2025	Containers with Podman [SO 2025-2]

Agenda

1. Sobre la practica (Enunciado)

<https://github.com/udea-so/SO-Lab3-20252>

2. Recursos (Entender antes de empezar)

https://github.com/udea-so/sesiones_lab_2025-2/tree/main/lab3/sesion-4

3. Conceptos importantes:

* Matemáticas:

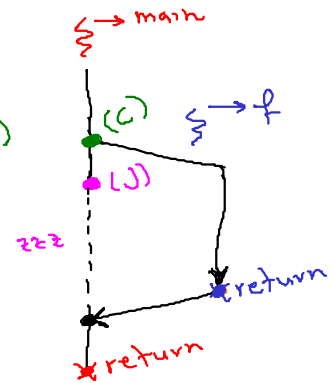
1. Regla del punto medio

* API (threads):

1. Sincronización: `-pthread_create(c)`

`-pthread_join(j)`

2. Como pasar argumentos a un hilo.



```
#include <pthread.h>

int
pthread_create(      pthread_t*      thread,
                    const pthread_attr_t* attr,
                    void*            (*start_routine)(void*),
                    void*            arg);
```

<https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-api>

Threads API

Some simple examples of how to use POSIX thread APIs.

Relevant files:

- `thread_create.c`: A simple thread creation program, with args passed to the thread.
- `thread_create_with_return_args.c`: Return values from the thread to the parent.
- `thread_create_simple_args.c`: Simpler argument/return value passing for the lazy.

Type `make` to build; each file `foo.c` generates an executable `foo` which you can then run, e.g.:

```
prompt> ./thread_create
```

```
#include <pthread.h>

typedef struct __myarg_t {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc;

    myarg_t args;
    args.a = 10;
    args.b = 20;
    rc = pthread_create(&p, NULL, mythread, &args);
    ...
}
```

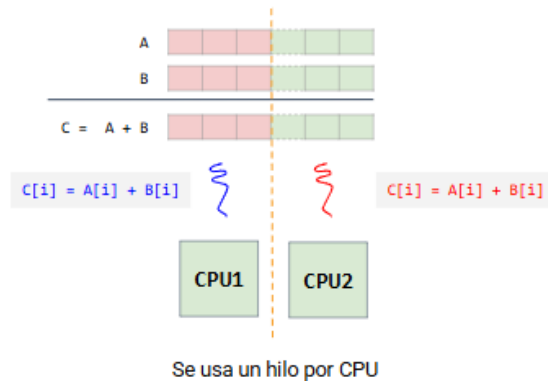
Ejemplos de clase (clase 14)

https://github.com/udea-so/apuntes_2025-2_SO-virtual/blob/main/clase_14/api-thread/t2.c

3. Tipos de paralelismo



Ejemplo: Suma de vectores



https://github.com/udea-so/sesiones_lab_2025-2/tree/main/lab3/sesion-4/examples/parte2

Paralelismo y Medición de Desempeño

Análisis de Speedup en la Suma de Vectores con Pthreads

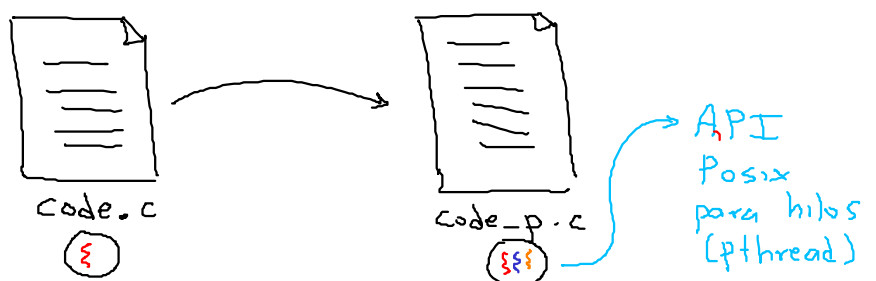
Ahora si empecemos...

1. Sobre la practica:

- 2 Problemas {
1. Cálculo numérico de π :
$$\int_0^1 \frac{4}{1+x^2} dx = \pi$$

PC: Cálculo se hace usando el Método del Punto Medio.
 2. Serie Fibonacci:
$$f_n = \begin{cases} 0 & ; \text{ si } n=0 \\ 1 & ; \text{ si } n=1 \\ f_{n-1} + f_{n-2} & ; \text{ si } n \geq 2 \end{cases}$$

Objetivo: Paralelización de código.

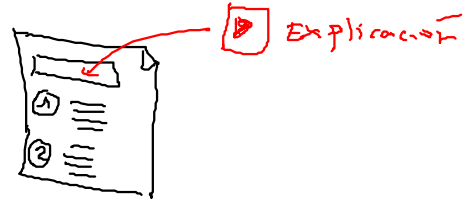


① Paralelizar código



② Informe

analysis.ipynb

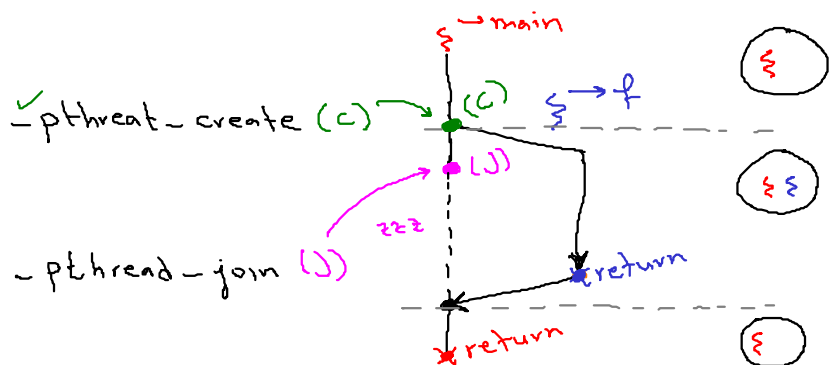


2. Recursos para entender por donde empezar

*API (threads):

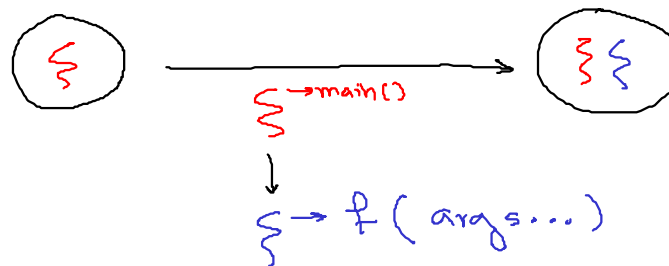
1. Sincronización: $\checkmark$ pthread_create (c)

pthread_join (j)



2. Patron fork-join

3. Como pasar argumentos a un hilo.



```
#include <pthread.h>

int
pthread_create(      pthread_t*      thread,
                    const pthread_attr_t* attr,
                    void*          (*start_routine)(void*),
                    void*          arg);
```

<https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-api>

wrapper (envoltorio)

```
#include <pthread.h>

typedef struct __myarg_t {
    int a;
    int b;
} myarg_t;

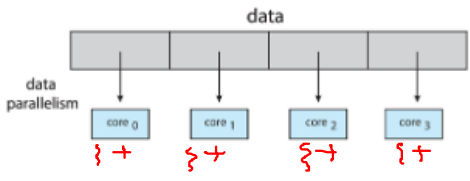
void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc;

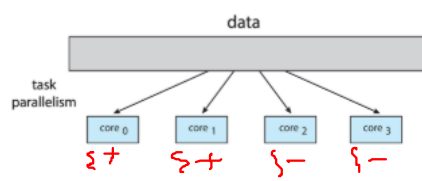
    myarg_t args;
    args.a = 10;
    args.b = 20;
    rc = pthread_create(&p, NULL, mythread, &args);
    ...
}
```

3. Tipos de paralelismo

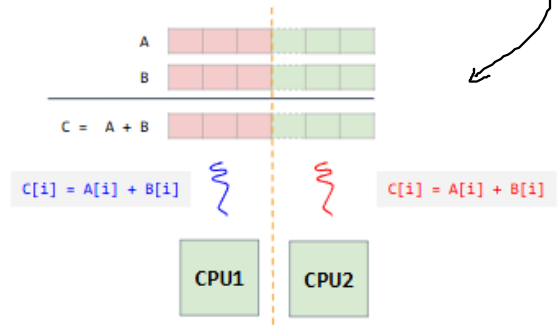
a. Paralelismo de Datos



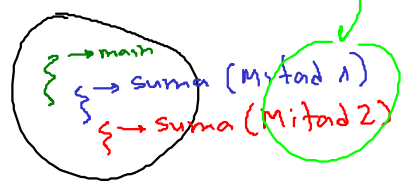
b. Paralelismo de tareas.



Ejemplo: Suma de vectores



Paralelismo de datos Argumentos



Se usa un hilo por CPU

$$\frac{103}{4} = 25.75$$

Serial

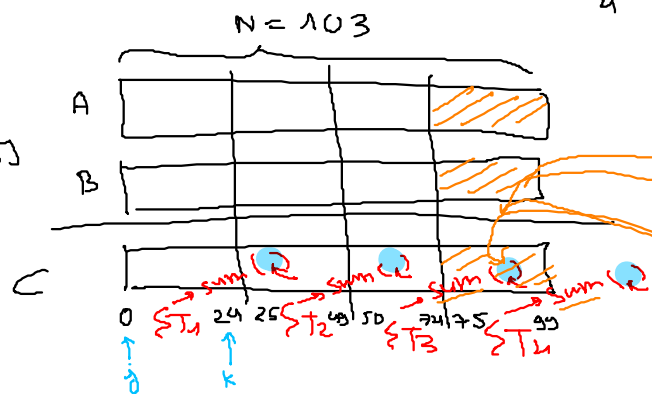
$$C_i = \sum_{i=0}^{N-1} [A_i + B_i]$$

↑
sum

Paralelo

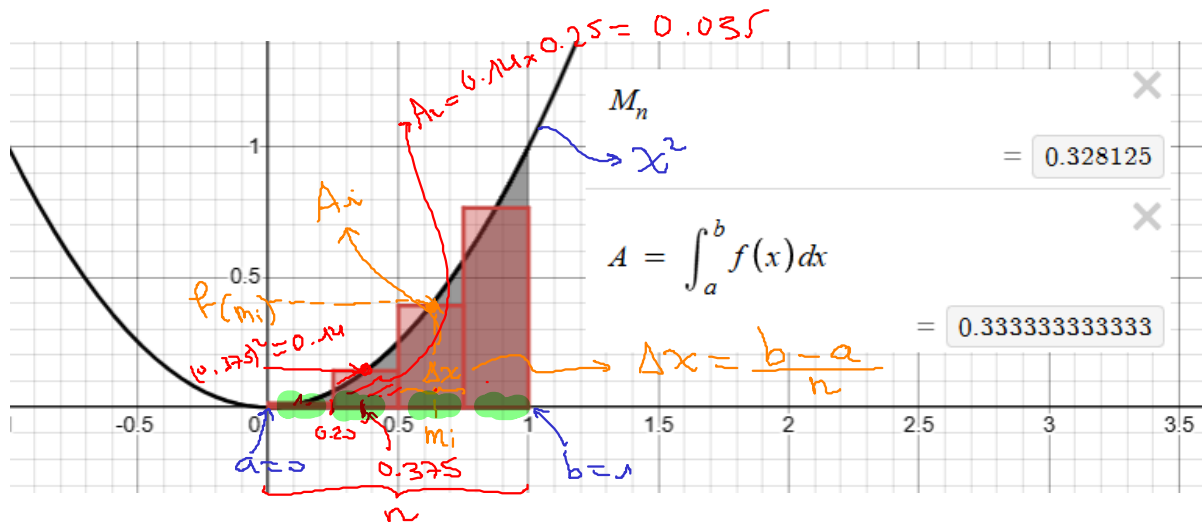
$T = 4$
 $N = 103$
 $C_i = \sum_{i=j}^k [A_i + B_i]$
 id

hilo	j	k
T1	0	24
T2	25	49
T3	50	74
T4	75	102



3. Conceptos importantes

<https://www.desmos.com/calculator/uaq1aiknfh>



Calcular el area bajo la curva:

Calculo 2:

$$A = \int_a^b f(x) dx$$

$$A = \int_0^1 x^2 dx = \left. \frac{x^3}{3} \right|_0^1 = \frac{1}{3} = 0.33$$

Calculo 2: Regla del punto medio

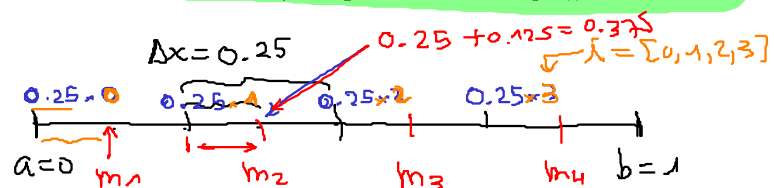
$$A = \sum_{i=1}^n A_i$$

$$A_i = b_i \times h_i = \Delta x \times f(m_i)$$

$$A_i = \left(\frac{b-a}{n} \right) \times f(m_i)$$

$$M_n = \sum_{i=1}^n A_i = \sum_{i=1}^n f(m_i) \Delta x$$

$$A = \lim_{n \rightarrow \infty} M_n = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(m_i) \Delta x$$



$$n = 4 \rightarrow \Delta x = \frac{b-a}{n} = \frac{1-0}{4} = 0.25$$

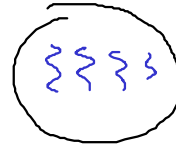
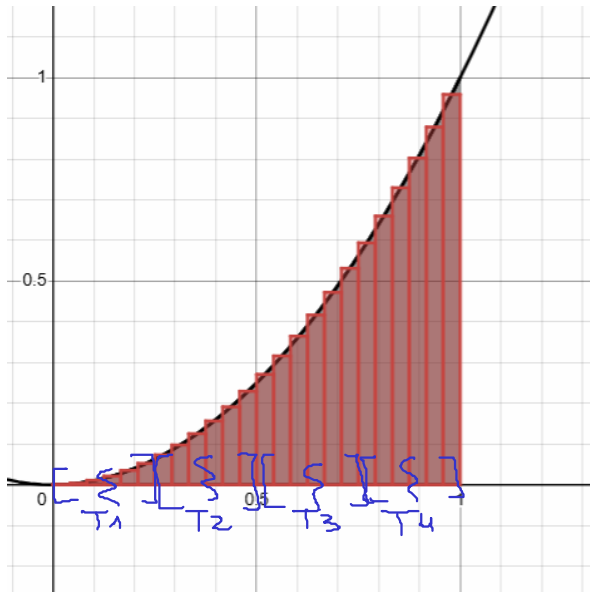
$$\Delta m = \frac{\Delta x}{2} = 0.125$$

$$m_i = \Delta x \cdot i + \Delta m$$

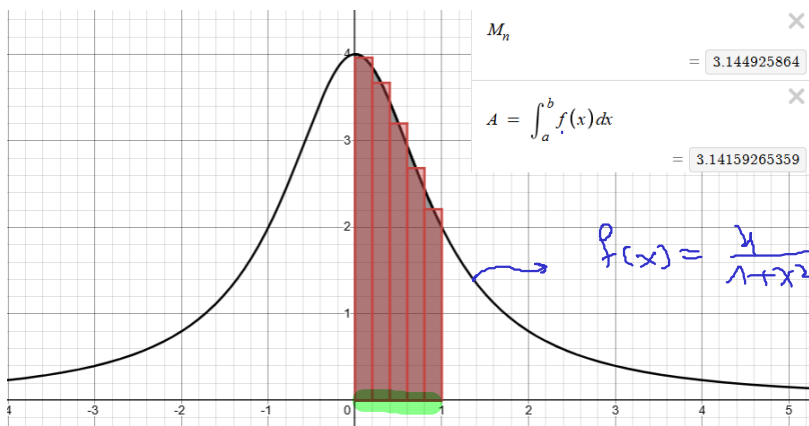
$$m_i = \Delta x \cdot i + \frac{\Delta x}{2}$$

$$m_i = \Delta x (i + 0.5)$$

$$h_i = f(x = m_i)$$



<https://www.desmos.com/calculator/bqkgkjaimm>



$$M_n = 3.144925864$$

$$A = \int_a^b f(x) dx = 3.14159265359$$

$$f(x) = \frac{1}{1+x^2}$$

$$M_n = \sum_{i=1}^n f(m_i) \Delta x$$

```
// extracto de pi.c
double CalcPi(int n)
{
    const double fH = 1.0 / (double) n;
    double fSum = 0.0;
    double fX;
    int i;

    // Bucle principal para paralelizar
    for (i = 0; i < n; i += 1)
    {
        fX = fH * ((double)i + 0.5);
        fSum += f(fX);
    }
    return fH * fSum;
}
```